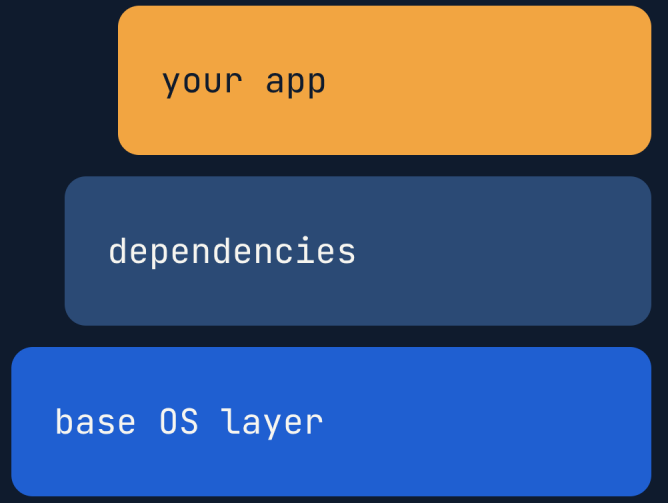
$ docker run workshop

Containers, hands-on

From namespaces to a running Postgres — with Docker and Podman



PRACTICAL WORKSHOP · ~100 MINUTES · [AHMAD KARIMI] · [2026-10-20]

BEFORE WE START

Setup check

1 · Install one engine

- macOS / Windows: Docker Desktop or Podman Desktop
- Linux: Docker Engine or Podman

Then check it answers:

```
$ docker version
$ podman version
```

2 · Pre-pull the images

```
$ docker pull hello-world
$ docker pull alpine
$ docker pull postgres:18
$ docker pull adminer
```

Registry slow or blocked? Bring a USB stick: **docker save -o images.tar postgres:18 alpine → docker load -i images.tar**

WARM-UP · EVERYONE TYPES

Your first container, in one line

```
$ docker run hello-world
```

You just downloaded a tiny program, packaged with everything it needs, and ran it — isolated from your machine.

What just happened?

1. The CLI asked the Docker daemon to run an image
2. Image not on your laptop → pulled from Docker Hub
3. A container was created from that image
4. It ran, printed its message, and exited

CONCEPTS · WHAT IS A CONTAINER?

A container is a normal Linux process — with blinders and a budget

BLINDERS

Namespaces

What it can **see**: its own processes, network, hostname, mounts and users.

BUDGET

cgroups

What it can **use**: CPU, memory, disk I/O, number of processes.

STARTING POINT

Image

What it **starts with**: a packaged filesystem mounted as its root “/”.

CONCEPTS · NAMESPACES

Each namespace hides one part of the system

Namespace	The container gets its own…
pid	process IDs — your app is PID 1
net	network interfaces, IP address, ports
mnt	view of the filesystem and mounts
uts	hostname
ipc	shared memory and message queues
user	user IDs — root inside can be nobody outside
cgroup	view of its own resource limits

Try it

```
$ docker run --rm alpine ps
PID   USER     COMMAND
    1 root     ps

$ docker run --rm alpine hostname
3f2a9c1d7b4e
```

Only one process visible, and a random hostname: the blinders at work.

CONCEPTS · CGROUPS

cgroups cap what a container can use

```
$ docker run -d --name busy \
    --memory=256m --cpus=0.5 \
    alpine sleep 3600

$ docker stats busy
```

No flags = no ceiling: one runaway container can starve the whole machine.

`--cpus=0.5`

At most half a CPU core; it gets slowed, not killed

`--memory=256m`

Go over it and the kernel kills the process (OOM)

`--pids-limit=100`

Caps the number of processes (stops fork bombs)

CONCEPTS · WHY CONTAINERS?

What containers fix

Same everywhere

Laptop, CI, server: one image, one behavior.

No version clashes

Postgres 16 and 18 side by side, no conflict.

Starts in seconds

No operating system to boot — just a process.

Lightweight

Shares the host kernel: megabytes, not gigabytes.

Disposable

Broke it? Delete it and start a fresh one.

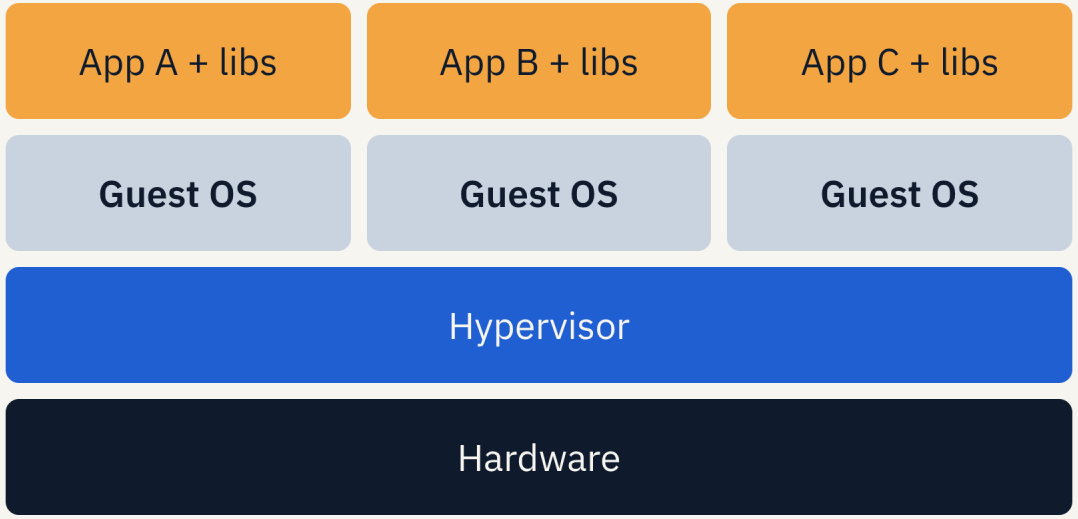
Easy to share

Push to a registry, pull it on any machine.

CONCEPTS · CONTAINERS VS VMS

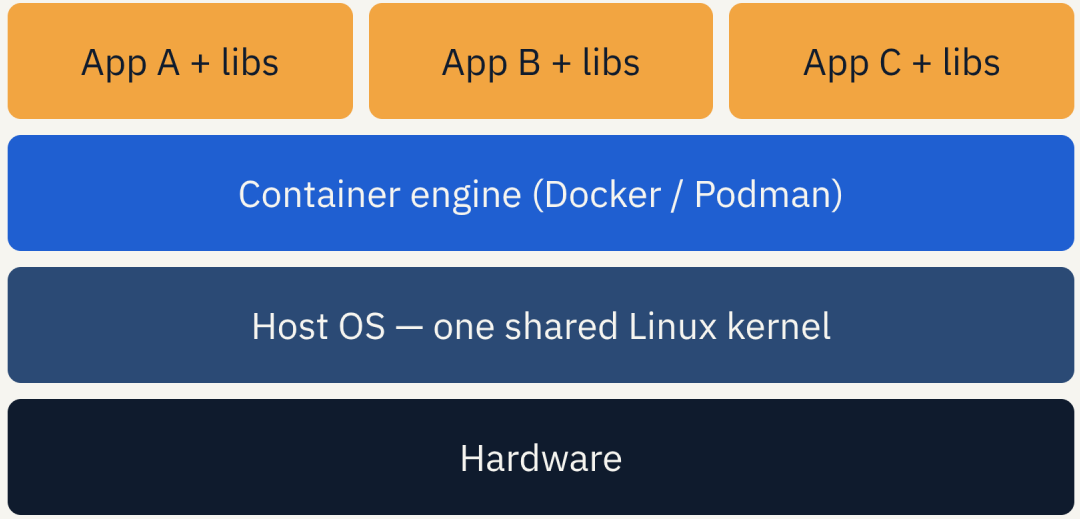
Same apps, two ways to isolate them

Virtual machines



Every VM boots its own full OS kernel.

Containers



All containers share the host’s kernel.

CONCEPTS · CONTAINERS VS VMS

The trade-off, side by side

	Virtual machine	Container
Isolation	Own kernel — a strong boundary	Shared kernel — process-level boundary
Start-up	Tens of seconds or more (boots an OS)	Usually under a few seconds
Size	Gigabytes	Megabytes to a few hundred MB
Runs	Any OS, e.g. Windows on a Linux host	Needs a compatible kernel (Linux containers need Linux)
Best for	Different OSes, strong multi-tenant isolation	Packaging and shipping apps, dev environments

Not rivals: in the cloud, your containers usually run **inside** VMs.

TOOLS · DOCKER & PODMAN

Engines do the plumbing for you

Build

`docker build`

Turn a Dockerfile into an image

Share

`docker pull / push`

Move images to and from registries

Run

`docker run / ps / rm`

Start, watch and remove containers

Wire up

`-p · -v · network`

Connect ports, storage and networks

Under the hood

Both follow the **OCI** standards, so an image built with Docker runs on Podman or Kubernetes — and the other way round. The container itself is started by a tiny low-level runtime (**runc** or **crun**) that sets up the namespaces and cgroups.

TOOLS · DOCKER & PODMAN

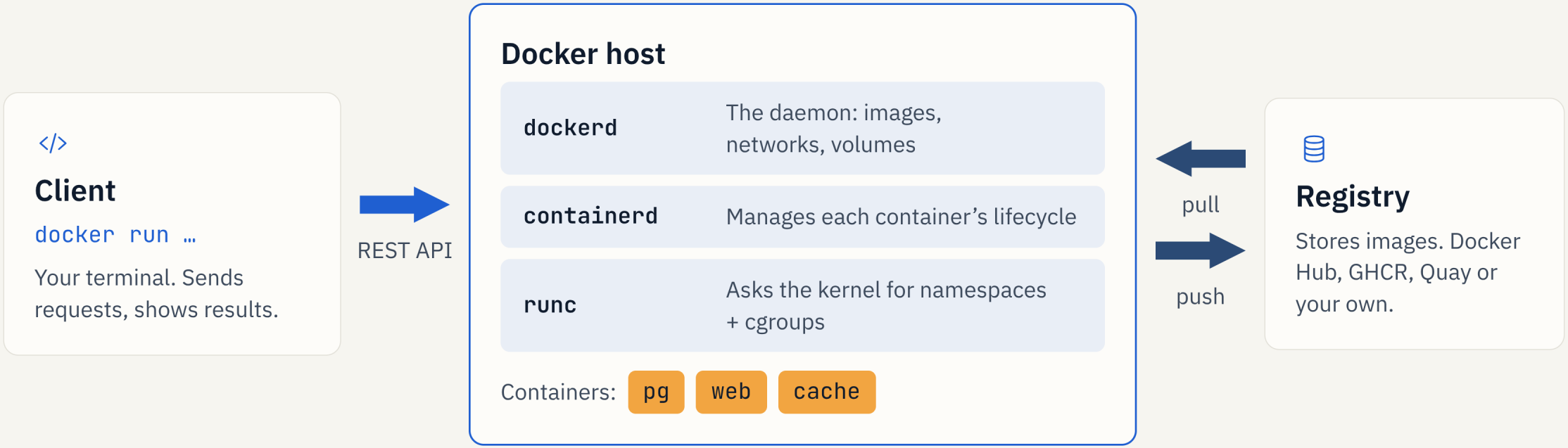
Docker vs Podman: same verbs, different design

	Docker	Podman
Design	Client talks to a background daemon	Daemonless — no background service
Privileges	Daemon runs as root; rootless optional	Rootless by default
CLI	docker run …	podman run …
Multi-container	docker compose	podman compose, plus pods
Desktop app	Docker Desktop (paid for big companies)	Podman Desktop (open source)

Learn one, you know the other: **alias docker=podman** covers nearly everything today.

TOOLS · DOCKER ARCHITECTURE

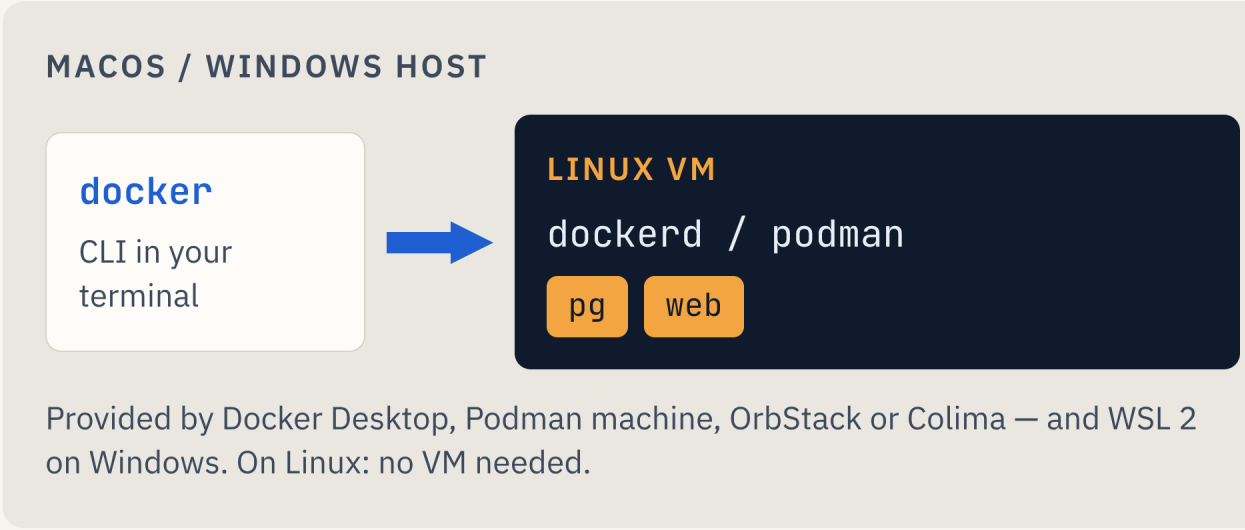
Client, daemon, registry



The CLI is only a messenger: “Cannot connect to the Docker daemon” means the daemon isn’t running.

TOOLS · MACOS & WINDOWS

On a Mac, a hidden Linux VM does the work



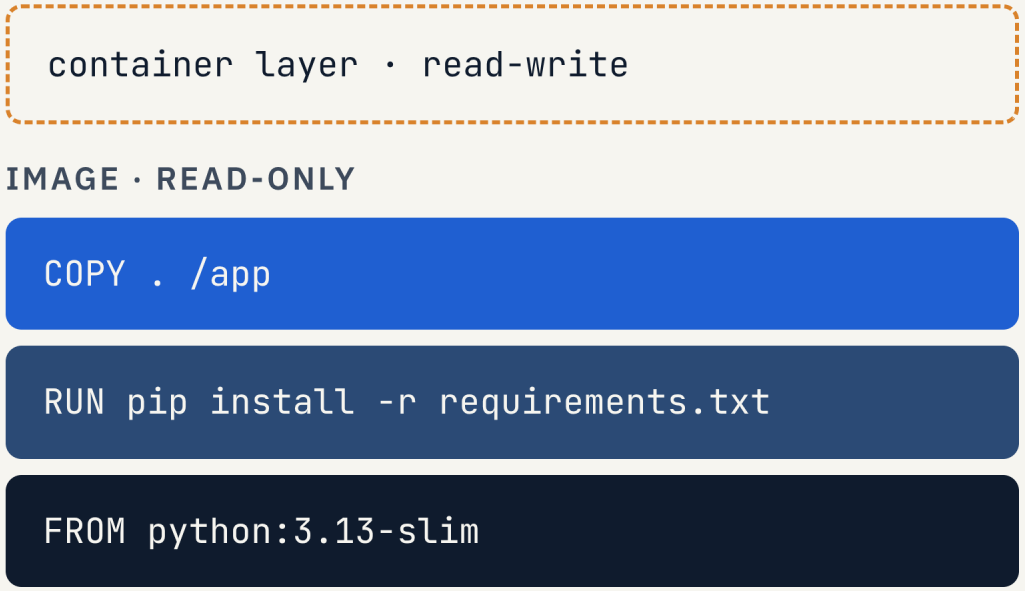
Provided by Docker Desktop, Podman machine, OrbStack or Colima — and WSL 2 on Windows. On Linux: no VM needed.

What this means for you

- Ports are forwarded from the VM to your localhost
- Bind mounts cross the VM boundary, so big folders are slower
- The VM has its own CPU and RAM limit — check the settings
- Apple Silicon wants arm64 images; others run emulated

IMAGES · LAYERS

An image is a stack of read-only layers




```
$ docker history postgres:18
```

One step, one layer

Each Dockerfile step that changes files adds a layer on top.

Layers are shared

Ten images on one base store that base once. Pulls fetch only missing layers.

Containers add a thin layer

Your writes go there — and vanish with the container.
Hence volumes.

IMAGES · NAMES & REGISTRIES

Reading an image name

```
docker.io/library/postgres:18-alpine
```

registry namespace repository tag

Short names expand

postgres means docker.io/library/postgres:latest

Pin your versions

latest is just a default label, not a promise of “newest”. Use **postgres:18**.

Digest = exact bytes

postgres@sha256:... never changes; tags can be moved.

IMAGES · DOCKERFILE

A Dockerfile is the recipe for an image

Dockerfile

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
ENV PORT=8000
EXPOSE 8000
CMD ["python", "app.py"]
```

Step	What it does
FROM	Base image to start from
WORKDIR	Like “cd” for the next steps (creates it)
COPY	Files from your folder into the image
RUN	Runs at build time, adds a layer
ENV	Sets an environment variable
EXPOSE	Documents a port; does not publish it
CMD	Default command at run time

IMAGES · BUILD & CACHE

Build, then order for the cache

```
$ docker build -t myapp:1.0 .    # "." = build context (this folder)
$ docker images
$ docker run --rm -p 8000:8000 myapp:1.0
```

SLOW ORDER

Every code edit reinstalls all deps

```
COPY . .
RUN pip install -r requirements.txt
```

FAST ORDER

Deps stay cached until they change

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

Rule: when a layer changes, it and **every layer after it** rebuild. Rarely-changing steps first.

RUNNING · CHEAT SHEET

The ten commands you'll use daily



(start brings a stopped one back)

`docker run IMAGE`	Create + start a container	`docker ps -a`	List all, incl. stopped
`docker logs -f NAME`	Follow its output	`docker exec -it NAME sh`	Open a shell inside it
`docker stop NAME`	Stop it; its files stay	`docker start NAME`	Start it again
`docker rm -f NAME`	Delete it and its files	`docker inspect NAME`	All the details, as JSON
`docker images`	List local images	`docker rmi IMAGE`	Delete an image

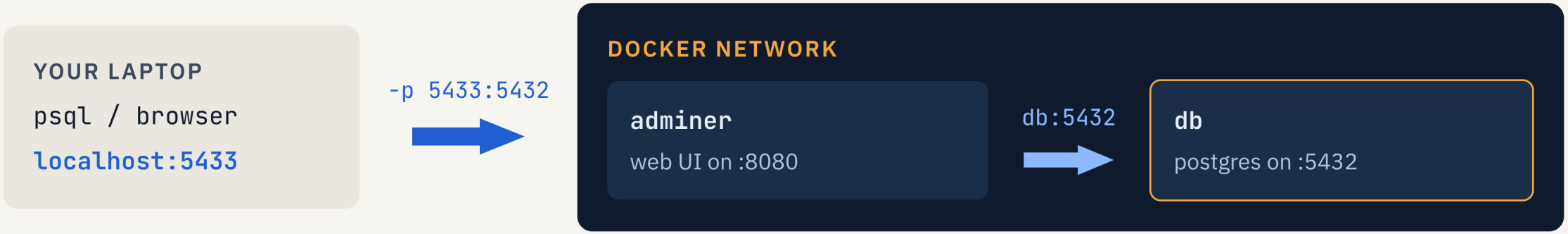
RUNNING · FLAGS

Anatomy of docker run

Flag	Meaning
-d	Detached: runs in the background
--name pg	A name to type instead of the random ID
-e KEY=value	Environment variable: config, passwords
-p 5433:5432	Publish a port: host 5433 → container 5432
-v pgdata:/var/lib/postgresql	Mount a volume at that path
--rm	Delete the container when it exits
-it	Interactive terminal, for shells and psql
postgres:18	The image — always after the flags

RUNNING · PORTS & NETWORKS

How traffic reaches a container



-p opens a door

From your laptop into one container port.

Names are addresses

On a shared network, containers reach each other by name.

Gotcha: localhost

Inside a container, localhost is that container itself.

RUNNING · STORAGE

Where container data should live

Named volume

```
-v pgdata:/var/lib/postgresql
```

Docker manages where it lives. Best for **databases**.

Survives docker rm: yes

Bind mount

```
-v "$(pwd)/src":/app
```

A folder from your laptop. Best for **live-editing code**.

Survives docker rm: yes — it's your folder

tmpfs

```
--tmpfs /tmp
```

Lives in RAM. Best for **scratch or secret** files.

Survives docker rm: no

No mount at all? Writes land in the container layer and are deleted with it.

RUNNING · VOLUMES

Working with volumes

```
$ docker volume create pgdata
$ docker volume ls
$ docker volume inspect pgdata
$ docker volume rm pgdata
$ docker volume prune  # unused only
```

Tip: **docker run -v pgdata:...** creates the volume for you if it doesn't exist yet.

GOTCHA · POSTGRES 18+

The mount path changed

postgres:18 and newer

`/var/lib/postgresql`

postgres:17 and older

~~`/var/lib/postgresql/data`~~

Most tutorials still show the old path. On 18 it can fail to start or leave your data outside your volume.

LAB 1 · PART A · HANDS-ON

Start a Postgres container

```
$ docker run -d --name pg \
    -e POSTGRES_PASSWORD=secret \
    -p 5432:5432 \
    -v pgdata:/var/lib/postgresql \
    postgres:18

$ docker ps
$ docker logs pg   # look for: ready to accept connections
```

CHECKPOINT pg shows as **Up** in docker ps. Port 5432 already taken by a local Postgres? Use -p 5433:5432.

LAB 1 · PART B · HANDS-ON

Connect and run a query

```
$ docker exec -it pg psql -U postgres

postgres=# CREATE TABLE students (
             id     serial PRIMARY KEY,
             name   text NOT NULL,
             joined date DEFAULT current_date);
postgres=# INSERT INTO students (name) VALUES ('Ada'), ('Linus');
postgres=# SELECT * FROM students;
postgres=# \q
```

Explore

\dt lists tables · **\d students** shows its columns

From your laptop (any SQL client)

localhost:5432 · postgres / secret

LAB 2 · 8 MIN · HANDS-ON

Destroy the container, keep the data

1. **Delete** the container

```
$ docker rm -f pg
```

2. **Recreate** it with the exact Lab 1 command (press ↑)

```
$ docker run -d --name pg … -v pgdata:/var/lib/postgresql postgres:18
```

3. **Check**: Ada and Linus are still there

```
$ docker exec -it pg psql -U postgres -c "SELECT * FROM students;"
```

4. **Break it on purpose**: repeat with a new volume — the table is gone

```
$ docker run … -v pgdata2:/var/lib/postgresql postgres:18
```

LAB 3 · 10 MIN · HANDS-ON

Build your own database image

init.sql

```
CREATE TABLE courses (
  id    serial PRIMARY KEY,
  title text NOT NULL
);
INSERT INTO courses (title) VALUES
  ('Containers 101'),
  ('SQL basics');
```

Dockerfile

```
# start from the official image
FROM postgres:18

# files here run on first start
COPY init.sql /docker-entrypoint-initdb.d/
```

```
$ docker build -t school-db:1.0 .
$ docker run -d --name school -e POSTGRES_PASSWORD=secret -p 5433:5432 school-db:1.0
$ docker exec -it school psql -U postgres -c "SELECT * FROM courses;"
```

Init scripts run **only when the data directory is empty** — a fresh container, not an old volume.

BONUS · 10 MIN · IF TIME ALLOWS

Same setup, one file: Compose

compose.yaml

```yaml
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql
  adminer:
    image: adminer
    ports:
      - "8080:8080"
volumes:
  pgdata:
```

1 · Free port 5432 first

```
docker rm -f pg
```

2 · Start everything

```
docker compose up -d
```

3 · Open localhost:8080 and log in

System PostgreSQL · Server **db** · postgres / secret

4 · Tear down

```
docker compose down    # keeps data
docker compose down -v # deletes it
```

LABS · PODMAN EDITION

The same labs with Podman

```
$ podman machine init   # macOS / Windows only, once
$ podman machine start
$ podman run -d --name pg -e POSTGRES_PASSWORD=secret \
    -p 5432:5432 -v pgdata:/var/lib/postgresql \
    docker.io/library/postgres:18
$ podman exec -it pg psql -U postgres
```

Full image names

docker.io/library/... skips the “which registry?” prompt.

Rootless ports

On Linux, host ports below 1024 need extra setup. Use 8080, not 80.

Pods

podman pod create groups containers, Kubernetes-style.

WRAP-UP · WHEN THINGS BREAK

Troubleshooting cheat sheet

You see…	Do this
Cannot connect to the Docker daemon	Start Docker Desktop, or podman machine start
Port is already allocated / address in use	Something uses 5432 → map -p 5433:5432
Exits: superuser password is not specified	Add -e POSTGRES_PASSWORD=…
Mount error on /var/lib/postgresql/data	On postgres:18+, mount /var/lib/postgresql
init.sql didn’t run	Volume was already initialized → use a new one
Pull is slow or times out	Use pre-pulled images or docker load -i images.tar
“It worked a minute ago”	docker ps -a: is it still running?
Anything else	docker logs NAME — read the last lines first

IF YOU REMEMBER THREE WORDS

Image	the recipe: read-only layers
Container	a running instance: an isolated process
Volume	the data that outlives the container

WRAP-UP

Clean up, then keep going

Free your disk

```
$ docker rm -f pg school
$ docker compose down -v
$ docker volume rm pgdata pgdata2
$ docker system prune
```

Learn next

- Multi-stage builds for small images
- Running as a non-root user
- Scanning images for vulnerabilities
- Kubernetes: containers at scale

Questions?

[your email or handle]